ADAM Optimizer: Theory, Implementation, and Benchmark Analysis

Research Documentation

Jasurbek Odilov
Research Assistant

Muhammad Usama Irfan
Research Assistant

Academic Supervisor: Prof. Dr. rer. nat. Tom Lahmer
Professorship of Stochastics and Optimization, Institute of Structural Mechanics (ISM)
Bauhaus-Universität Weimar — Introduction to Optimization

November 7, 2026

1 Theoretical Foundations of the ADAM Optimizer

1.1 Algorithm Overview

Adam (Adaptive Moment Estimation) [4] is an algorithm for first-order gradient-based optimization of stochastic objective functions. The method is based on adaptive estimates of lower-order moments of the gradients. It computes individual adaptive learning rates for different parameters from estimates of first (the mean) and second (the uncentered variance) moments of the gradients.

Adam combines the advantages of two established first-order optimization methods:

- **AdaGrad** [3]: which works well with sparse gradients by scaling learning rates inversely proportional to the square root of historical squared gradients.
- **RMSProp** [12]: which works well in on-line and non-stationary settings through exponential moving averages.

1.2 Mathematical Formulation and Update Rules

Let $f(w)$ be the differentiable objective function with parameter vector $w \in \mathbb{R}^D$. We initialize the standard default hyperparameters:

- Stepsize (learning rate): $\alpha = 0.001$
- Exponential decay rates: $\beta_1 = 0.9$, $\beta_2 = 0.999$
- Smoothing term to prevent division by zero: $\varepsilon = 10^{-8}$
- Convergence tolerance threshold: $\tau = 10^{-5}$ (checking if the gradient norm $\|\nabla f(w_t)\|_2$ drops below τ)
- First moment vector: $m_0 = \mathbf{0}$
- Second moment vector: $v_0 = \mathbf{0}$
- Timestep: $t = 0$
- Initial parameter vector: w_0

Setting first moment vector $m_0 = \mathbf{0}$, second moment vector $v_0 = \mathbf{0}$, timestep $t = 0$, and initial parameter vector w_0, the optimization loop executes while $\|\nabla f(w_t)\|_2 > \tau$ as follows:

1. Increment the iteration counter:
$$t \leftarrow t + 1 \tag{1}$$

2. Get gradients with respect to the stochastic objective at timestep t:
$$g_t \leftarrow \nabla f_t(w_{t-1}) \tag{2}$$

3. Update exponential moving average of the gradient (biased first moment estimate):
$$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{3}$$

4. Update squared gradients (biased second raw moment estimate):
$$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2)(g_t \odot g_t) \tag{4}$$

These moving averages are initialized as vectors of zeros ($m_0 = \mathbf{0}, v_0 = \mathbf{0}$), leading to moment estimates that are biased towards zero, especially during initial timesteps, and especially when the decay rates are small (β_1, β_2 close to 1). This initialization bias is counteracted by computing bias-corrected estimates $\hat{m}_t$ and $\hat{v}_t$:

5. Compute bias-corrected first moment estimate:
$$\hat{m}_t \leftarrow \frac{m_t}{1 - \beta_1^t} \tag{5}$$

6. Compute bias-corrected second raw moment estimate:
$$\hat{v}_t \leftarrow \frac{v_t}{1 - \beta_2^t} \tag{6}$$

7. Update parameters:
$$w_t \leftarrow w_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \tag{7}$$

This operation computes individual adaptive learning rates for different parameters. By using the square root of the uncentered variance ($\sqrt{\hat{v}_t}$) in the divisor, the step size exhibits invariance to a diagonal rescaling of the gradients, scaling updates efficiently along the geometry of the objective function.

Computational Efficiency Optimization: Note that the efficiency of the algorithm can, at the expense of clarity, be improved upon by changing the order of computation, e.g. by replacing the last three lines in the loop with the following time-dependent scalar stepsize:

$$\alpha_t = \alpha \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t} \tag{8}$$

$$w_t \leftarrow w_{t-1} - \alpha_t \frac{m_t}{\sqrt{v_t} + \hat{\varepsilon}} \quad \text{where } \hat{\varepsilon} = \varepsilon \sqrt{1 - \beta_2^t} \tag{9}$$

This version saves execution time because the computer calculates a scalar factor α_t first, rather than applying element-wise vector divisions to the momentum arrays m_t and v_t.

2 MATLAB Implementation

The MATLAB implementation consists of three main scripts:

- `adam_optimizer.m`: Core ADAM optimizer function.
- `adam_runner_rosenbrock_func.m`: Runner script for benchmark testing.
- `visualize.m`: Plotting script for trajectories and convergence curves.

Algorithm 1 ADAM Optimizer Engine (`adam_optimizer.m`)

Require: Objective gradient $\nabla f(w)$, initial vector w_0, maximum iterations K, hyperparameters $(\alpha, \beta_1, \beta_2, \varepsilon, \tau)$

$w \leftarrow w_0, \quad m \leftarrow \mathbf{0}, \quad v \leftarrow \mathbf{0}, \quad t \leftarrow 0$
while $\|\nabla f(w)\|_2 > \tau$ **and** $t < K$ **do**
  $t \leftarrow t + 1$
  $g \leftarrow \nabla f(w)$
  $m \leftarrow \beta_1 m + (1 - \beta_1) g$ ▷ Biased first moment
  $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$ ▷ Biased second raw moment
  $\hat{m} \leftarrow m / (1 - \beta_1^t)$ ▷ Bias-corrected first moment
  $\hat{v} \leftarrow v / (1 - \beta_2^t)$ ▷ Bias-corrected second moment
  $w \leftarrow w - \alpha \cdot \hat{m} \oslash (\sqrt{\hat{v}} + \varepsilon)$ ▷ Parameter update
  history.w$(:, t) \leftarrow w$
end while
return $w^* = w$, history

3 2D Benchmark Functions: Rosenbrock and Beale

To evaluate the optimizer's performance on non-convex landscapes with narrow, ill-conditioned valleys, we test ADAM on two standard 2D benchmarks: the Rosenbrock function and the Beale function.

3.1 Rosenbrock Function

Objective Function: The Rosenbrock function [10] is defined as:

$$f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2 \tag{10}$$

Analytical Gradient: Differentiating f with respect to x_1 and x_2 yields:

$$\frac{\partial f}{\partial x_1} = -2(1 - x_1) - 400x_1(x_2 - x_1^2) \tag{11}$$

$$\frac{\partial f}{\partial x_2} = 200(x_2 - x_1^2) \tag{12}$$

Global Minimum: The unique global minimum is located at $x^* = (1, 1)$ with $f(x^*) = 0$.

Table 1: Hyperparameter Setup for 2D Rosenbrock Benchmark

Parameter	**Value**
w_0	$(-1.5, 2.0)$
α	0.01
max_iter	15,000
β_1	0.9
β_2	0.999
ε	1×10^{-8}
tol	1×10^{-5}

Results & Verification: ADAM converged to the exact global minimum in 4,145 iterations, matching the Nelder-Mead solution from MATLAB's `fminsearch`.

Table 2: Verification of Solution on Rosenbrock Function

Method	x_1	x_2
Known minimum	1.0000	1.0000
ADAM optimizer	1.0000	1.0000
`fminsearch` verification	1.0000	1.0000

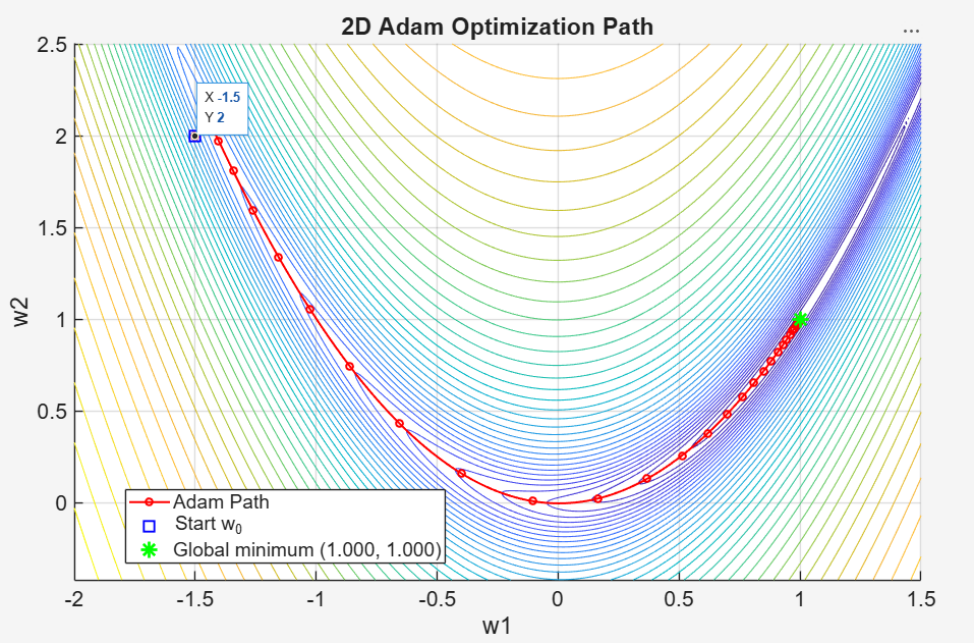


Figure 1: Optimization trajectory of ADAM on the 2D Rosenbrock contour.

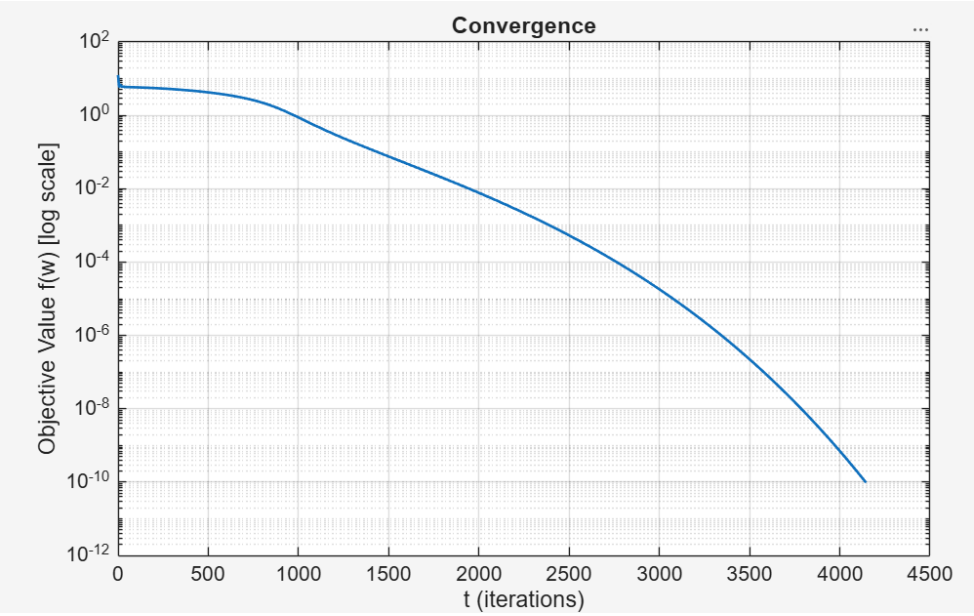


Figure 2: Convergence history of the objective value $f(w_t)$ on logarithmic scale for the 2D Rosenbrock benchmark.

Convergence Dynamics: ADAM quickly descends from the initial point $(-1.5, 2.0)$ into the parabolic ravine during the first 1,500 iterations. It then steadily traverses the curved valley floor without oscillating across the steep walls, reaching the minimum $(1, 1)$ with high precision.

3.2 Beale Function

Objective Function: The Beale function is defined as:

$$f(x_1, x_2) = (1.5 - x_1 + x_1 x_2)^2 + (2.25 - x_1 + x_1 x_2^2)^2 + (2.625 - x_1 + x_1 x_2^3)^2 \tag{13}$$

Analytical Gradient: Differentiating f with respect to x_1 and x_2 yields:

$$\frac{\partial f}{\partial x_1} = 2x_1 x_2^6 + 2x_1 x_2^4 - 4x_1 x_2^3 - 2x_1 x_2^2 - 4x_1 x_2 + 6x_1 + \frac{21}{4} x_2^3 + \frac{9}{2} x_2^2 + 3x_2 - \frac{51}{4} \tag{14}$$

$$\frac{\partial f}{\partial x_2} = 6x_1^2 x_2^5 + 4x_1^2 x_2^3 - 6x_1^2 x_2^2 - 2x_1^2 x_2 - 2x_1^2 + \frac{63}{4} x_1 x_2^2 + 9x_1 x_2 + 3x_1 \tag{15}$$

Global Minimum: The unique global minimum is located at $x^* = (3, 0.5)$ with $f(x^*) = 0$.

Table 3: Hyperparameter Setup for 2D Beale Benchmark

Parameter	**Value**
w_0	$(1, 0)$
α	0.01
max_iter	15,000
β_1	0.9
β_2	0.999
ε	1×10^{-8}
tol	1×10^{-5}

Results & Verification: ADAM converged to the global minimum in 2,491 iterations, matching the Nelder-Mead benchmark from MATLAB's `fminsearch`.

Table 4: Verification of Solution on Beale Function

Method	x_1	x_2
Known minimum	3.0000	0.5000
ADAM optimizer	3.0000	0.5000
`fminsearch` verification	3.0000	0.5000

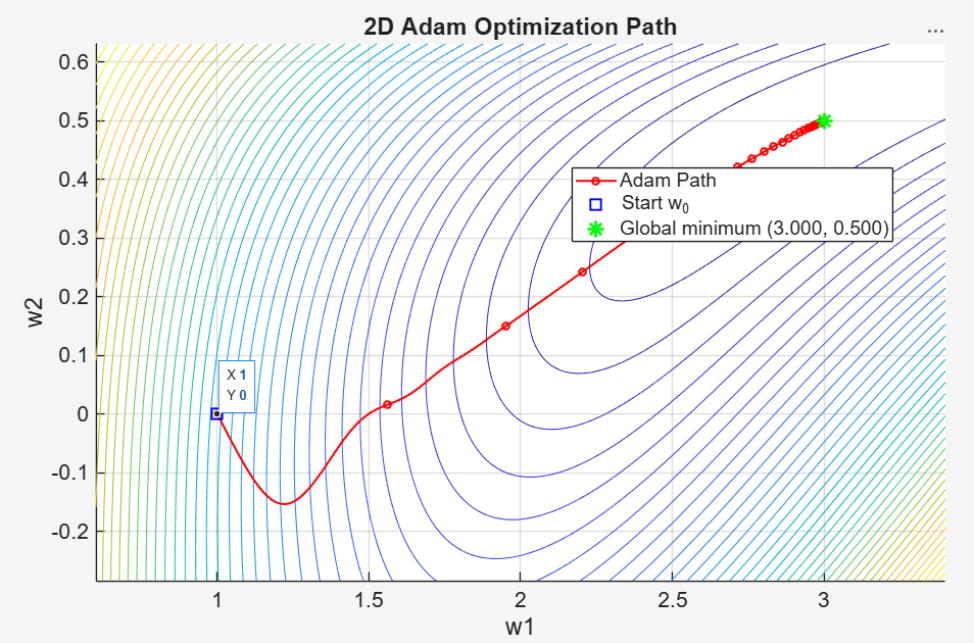


Figure 3: Optimization trajectory of ADAM on the 2D Beale contour.

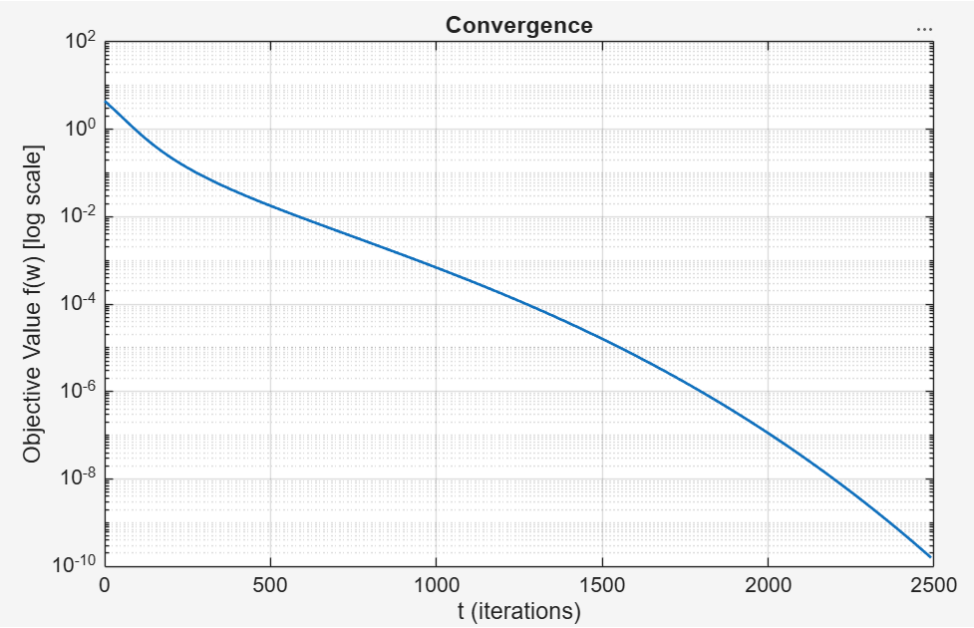


Figure 4: Convergence history of the objective value $f(w_t)$ on logarithmic scale for the 2D Beale benchmark.

Convergence Dynamics: Starting from $(1, 0)$, ADAM smoothly navigates the sharp contours and extended flat regions of the Beale surface, achieving steady logarithmic loss reduction down to the tolerance threshold at $(3, 0.5)$.

4 Comparative Analysis: ADAM vs. Steepest Descent and Newton's Method

To understand how ADAM behaves on curved, ill-conditioned valleys, we compare its trajectory on the 2D Rosenbrock function with two classical optimization methods:

- **Steepest Descent**: Standard first-order method updating along $w_t = w_{t-1} - \alpha \nabla f(w_{t-1})$.
- **Newton's Method**: Second-order method utilizing the exact inverse Hessian $w_t = w_{t-1} - H^{-1}(w_{t-1}) \nabla f(w_{t-1})$, with:

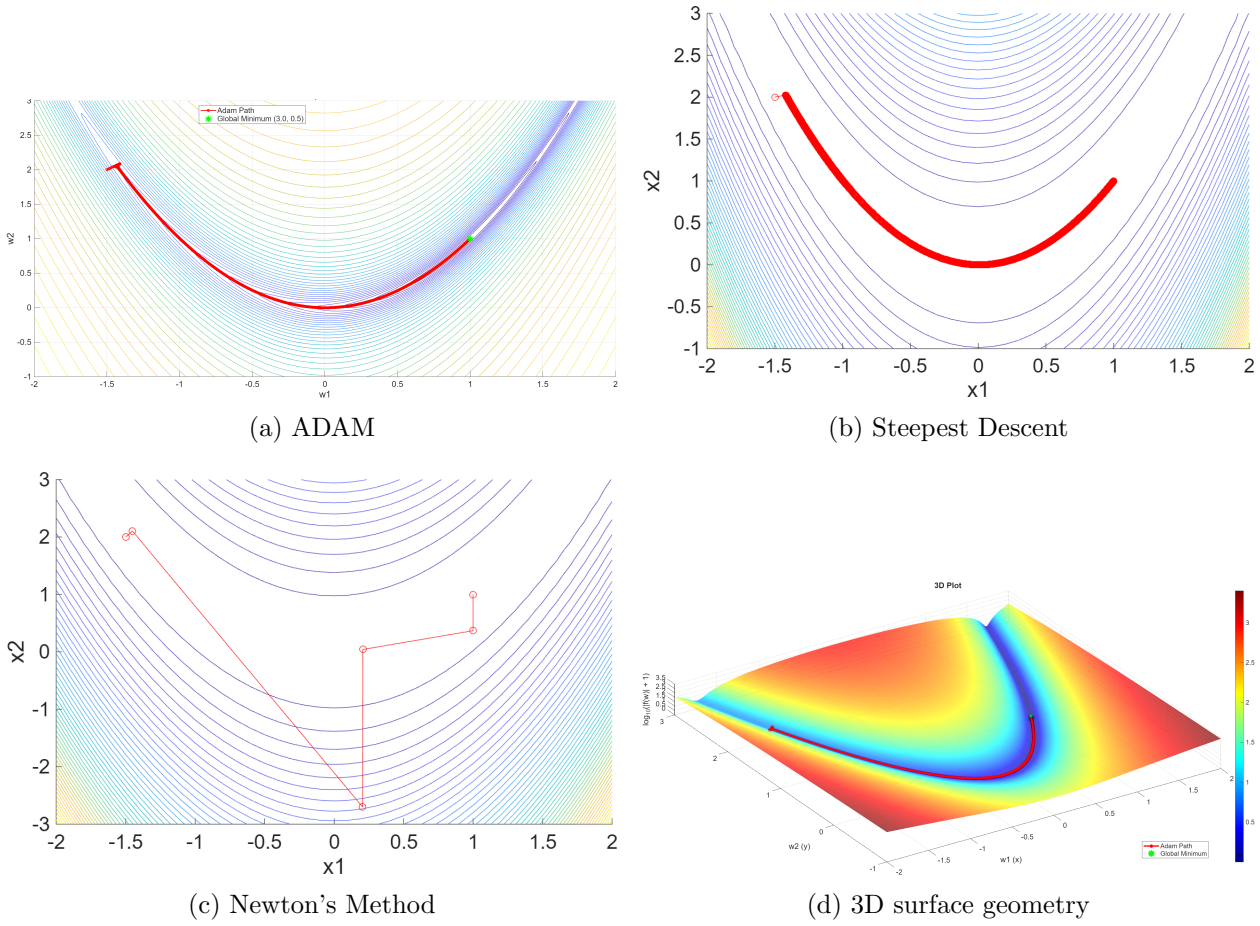
$$H(x_1, x_2) = \begin{bmatrix} 2 - 400x_2 + 1200x_1^2 & -400x_1 \\ -400x_1 & 200 \end{bmatrix} \tag{16}$$

- **ADAM**: First-order adaptive method combining momentum with coordinate-wise learning rate scaling.

All methods were evaluated from the same initial point $w_0 = (-1.5, 2.0)$ with a convergence tolerance of $\tau = 10^{-5}$.

Table 5: Iteration Count Comparison on 2D Rosenbrock Function

Optimization Method	**Iterations to Convergence**
ADAM	4,145
Steepest Descent	13,821
Newton's Method	5



(a) ADAM

(b) Steepest Descent

(c) Newton's Method

(d) 3D surface geometry

Figure 5: Optimization trajectories of ADAM, Steepest Descent, and Newton's Method on the 2D Rosenbrock contour, alongside the 3D surface geometry.

- **Steepest Descent** struggles because the gradient vectors are nearly perpendicular to the long valley floor. As a result, the iterates bounce back and forth between the steep valley walls, making slow progress along the bottom and taking 13,821 iterations to converge.
- **ADAM** uses momentum to dampen these cross-valley oscillations while accelerating along the valley floor. Individual coordinate scaling adapts step sizes along each direction, navigating the curve smoothly in 4,145 iterations without requiring second derivatives.
- **Newton's Method** uses exact curvature information from the Hessian matrix, finding the global minimum in only 5 iterations. However, forming and inverting the $D \times D$ Hessian requires $\mathcal{O}(D^3)$ operations per step, which becomes computationally impractical for high-dimensional problems.

5 Medium-Scale Benchmark: 100D Extended Rosenbrock

5.1 Formulation and Gradient Derivation

Moving beyond 2D tests, we evaluate ADAM on a medium-scale benchmark by extending the Rosenbrock function to $D = 100$ design variables:

$$f(w) = \sum_{i=1}^{D-1} \left[100(w_{i+1} - w_i^2)^2 + (1 - w_i)^2\right], \quad w \in \mathbb{R}^D \tag{17}$$

The global minimum is located at $w^* = (1, 1, \ldots, 1)^T$ with $f(w^*) = 0$.
The exact analytical gradient components $\nabla f(w) = [\frac{\partial f}{\partial w_1}, \ldots, \frac{\partial f}{\partial w_D}]^T$ are derived as:

$$\frac{\partial f}{\partial w_1} = -400w_1(w_2 - w_1^2) - 2(1 - w_1) \tag{18}$$

$$\frac{\partial f}{\partial w_i} = 200(w_i - w_{i-1}^2) - 400w_i(w_{i+1} - w_i^2) - 2(1 - w_i), \quad \text{for } 2 \leq i \leq D - 1 \tag{19}$$

$$\frac{\partial f}{\partial w_D} = 200(w_D - w_{D-1}^2) \tag{20}$$

5.2 Simulation Setup

The experiment is executed via `adam_runner_100D_extended_rosenbrock_func.m` with the following configuration:

- Dimension: $D = 100$
- Initial Guess: $w_0 = [-1.5, 2.0, -1.5, 2.0, \ldots]^T \in \mathbb{R}^{100}$
- Hyperparameters: $\alpha = 0.005$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\tau = 10^{-5}$
- Maximum Iterations: 25,000

5.3 Empirical Results & Convergence Dynamics

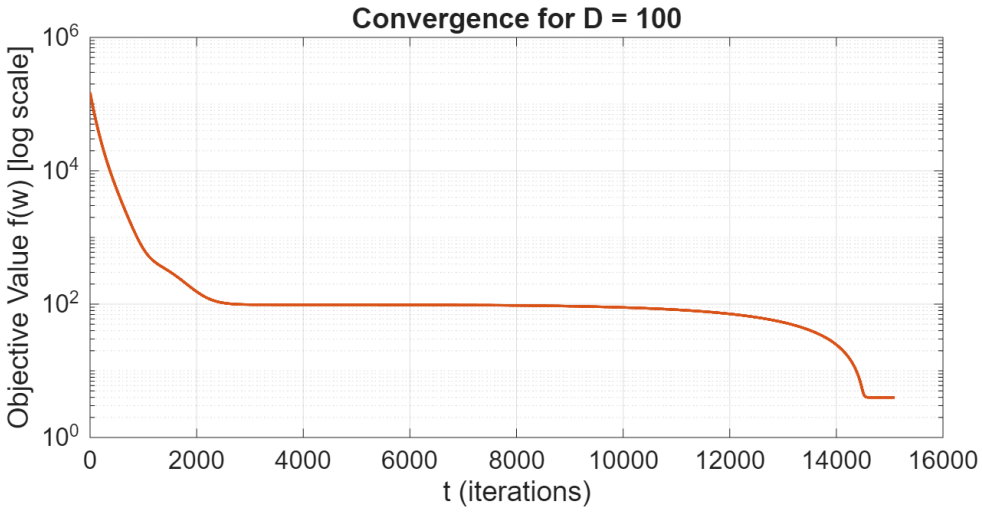


Figure 6: Objective function convergence history on logarithmic scale for the 100D Extended Rosenbrock benchmark.

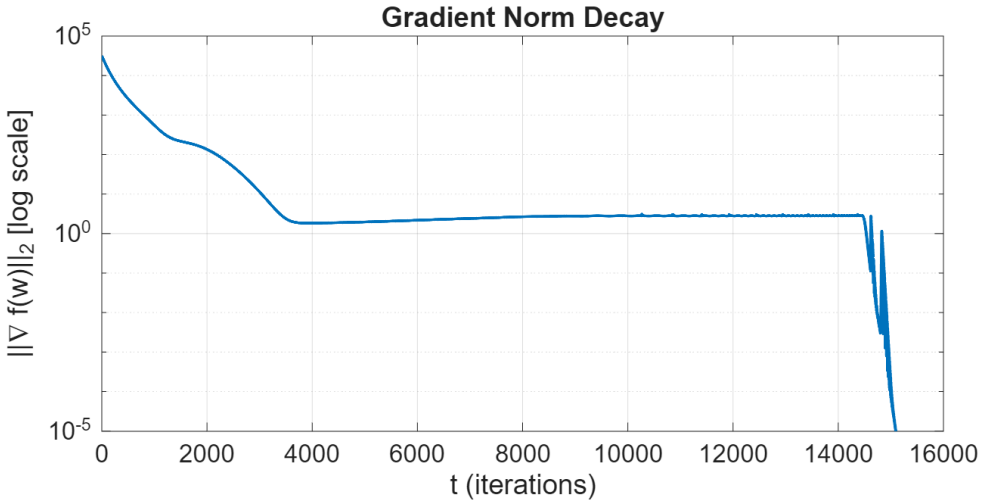


Figure 7: Gradient norm decay $\|\nabla f(w_t)\|_2$ during optimization of the 100D Extended Rosenbrock problem.

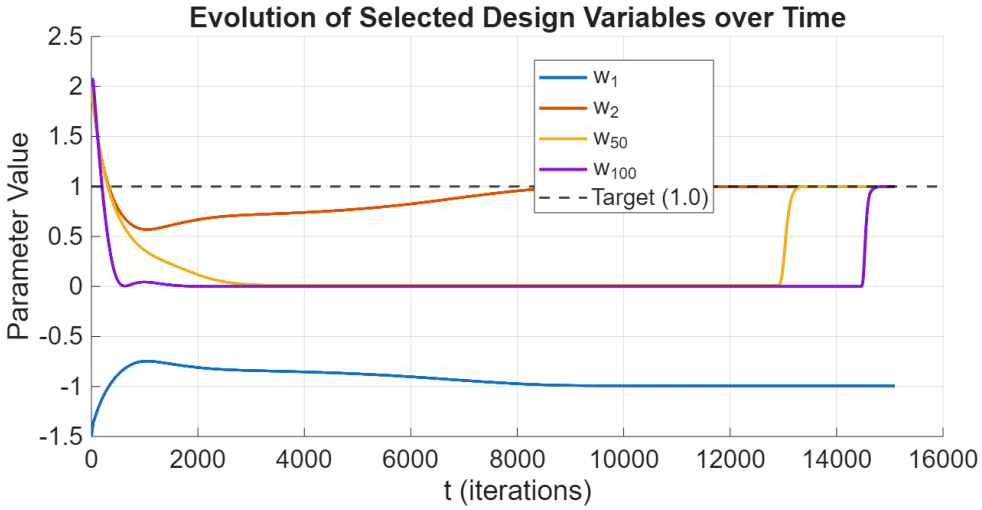


Figure 8: Evolution of selected design variables $(w_1, w_2, w_{50}, w_{100})$ during optimization of the 100D Extended Rosenbrock benchmark.

Table 6: Empirical Convergence Metrics for 100D Extended Rosenbrock Optimization

Metric	**Observed Value**
Iterations to Convergence	15,094
Computation Time	0.96 seconds
Final Objective Value $f(w^*)$	3.986624×10^0
ℓ_2-Distance to Global Minimum $\|w^* - w_{\text{global}}\|_2$	1.993290×10^0
Gradient Norm at Convergence $\|\nabla f(w^*)\|_2$	9.851188×10^{-6} (satisfies $\tau \leq 10^{-5}$)
Design Variable Range $[\min(w^*), \max(w^*)]$	$[-0.9933, 1.0000]$

- **Objective Decay:** As shown in Figure 6, the objective function exhibits a steep initial descent followed by steady logarithmic progress as ADAM traverses the 100-dimensional valley, reaching $f(w^*) = 3.9866$ within 0.96 seconds (15,094 iterations).

- **Gradient Norm:** Figure 7 confirms stable convergence to the specified gradient norm tolerance $\|\nabla f(w)\|_2 = 9.85 \times 10^{-6} \leq 10^{-5}$ without oscillations or numerical divergence.
- **Coordinate Dynamics:** Figure 8 tracks early (w_1, w_2), intermediate (w_{50}), and boundary (w_{100}) variables, illustrating coordinated convergence across all 100 dimensions.

5.4 Comparative Benchmark of Modern ADAM Variants

To investigate algorithmic behavior across different optimizer formulations, we evaluated six variants on the same 100-dimensional Extended Rosenbrock benchmark ($w_0 = [-1.5, 2.0, \dots]^T$, $\alpha = 0.005$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\tau = 10^{-5}$, max iterations 25,000). For the dual-trajectory twin variant, the second starting point was initialized symmetrically as $w_{02} = [2.0, -1.5, \dots]^T$. Table 7 summarizes the empirical performance metrics. Note that Table 7 reports pure optimizer loop execution times without visualization logging, where baseline ADAM runs in 0.11 s (compared to 0.96 s in Table 6 when recording full step-by-step diagnostic trajectories).

Table 7: Benchmark Comparison of ADAM Variants on 100D Extended Rosenbrock

Optimizer	**Iterations**	**Time (s)**	$f(w^*)$	$\|w^* - w^*_{\text{global}}\|_2$	$\|\nabla f(w^*)\|_2$
ADAM	15,094	0.11	3.9866	1.9933	9.85×10^{-6}
AdamW ($\lambda = 0.1$)	25,000	0.16	**0.0024**	**0.0527**	2.3474
AMSGrad	25,000	0.16	98.3683	9.9052	3.4744
ADAM-DG (Hypergradient)	**8,736**	**0.07**	3.9866	1.9933	9.18×10^{-6}
ADAM (Twin Gradient)	14,921	0.33	3.9866	1.9933	9.90×10^{-6}
Preconditioned ADAM	25,000	0.91	3.9874	1.9933	1.4066

Analysis of Convergence Characteristics:

- **Local Minimum Basin vs. Saddle Point (ADAM & ADAM-DG):** Both standard ADAM and ADAM-DG successfully satisfied the gradient convergence tolerance $\tau = 10^{-5}$, settling at $w_1^* = -0.9933$ and $w_i^* = 1.0000$ for $i \geq 2$, with $f(w^*) \approx 3.9866$. Spectral analysis confirms that all 100 eigenvalues of the Hessian at this point are strictly positive, ranging from $\lambda_{\min} \approx 0.499$ to $\lambda_{\max} \approx 1801.6$. This proves that the optimizers converged to a genuine, highly flat local minimum basin rather than stalling at a saddle point, matching the theoretical stationary point analysis of Kok and Sandrock [5].
- **Decoupled Weight Decay Mechanics (AdamW):** AdamW [6] reached the lowest objective value ($f = 0.0024$) and closest distance to the global minimum ($\|w^* - w^*_{\text{global}}\|_2 = 0.0527$). However, it reached the iteration limit because the gradient norm remained at 2.3474. In AdamW, parameter updates include a direct decay term $-\alpha\lambda w_t$ independent of the gradient. Consequently, equilibrium occurs where the objective gradient balances the regularization pull ($\nabla f(w) \approx \lambda w$), meaning $\|\nabla f(w)\|_2$ does not vanish to zero.
- **Denominator Saturation and Freezing (AMSGrad):** AMSGrad [9] failed to traverse the valley, stalling at $f = 98.3683$ with coordinates w_6 through w_{100} remaining near their starting values. Because AMSGrad enforces $\hat{v}_t = \max(\hat{v}_{t-1}, v_t)$, large initial gradient spikes permanently inflated the denominator, suppressing effective step sizes for the remainder of the run.
- **Dual-Trajectory Performance (ADAM with Twin Gradient):** The twin-modulated variant terminated in 14,921 iterations, slightly ahead of standard ADAM (15,094 iterations). Termination was triggered by Twin 1 satisfying gradient tolerance ($\|\nabla f(w_1)\|_2 \leq 10^{-5}$) at the local minimum plateau ($f = 3.9866$). Although Twin 2 reached the global basin

($f(w_2) = 1.24 \times 10^{-7}$), its gradient norm was 4.40×10^{-4}, causing the algorithm to select Twin 1 as w^*. Evaluating two gradients per step doubled total workload to 29,842 evaluations (0.33 s), showing that the twin coupling added substantial computational overhead without guaranteeing that the global basin point would have the smaller gradient norm.

6 Advanced Gradient Dynamics and Curvature Preconditioning

6.1 Double Gradient Descent via Hypergradient Adaptation

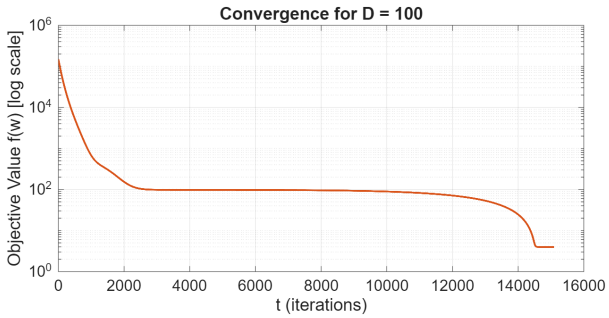
Standard ADAM relies on a fixed, manually tuned learning rate α. Double gradient descent (hypergradient descent) [1] formulates the stepsize α itself as an optimization parameter, updating it at each iteration via gradient descent on the objective function with respect to α:

$$\alpha_t = \alpha_{t-1} - \beta_h \frac{\partial f(w_{t-1})}{\partial \alpha_{t-1}} \tag{21}$$

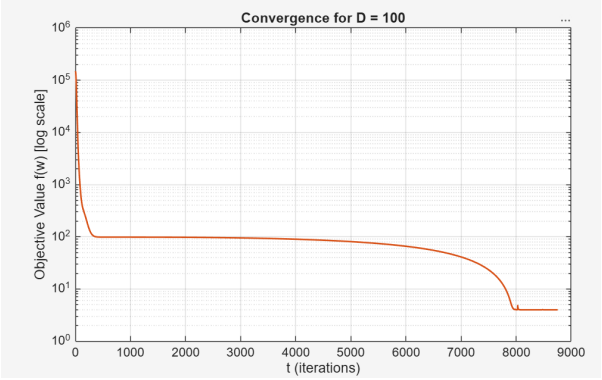
Applying the chain rule yields:

$$\frac{\partial f(w_{t-1})}{\partial \alpha_{t-1}} = \nabla f(w_{t-1})^T \frac{\partial w_{t-1}}{\partial \alpha_{t-1}} = -\nabla f(w_{t-1})^T u_{t-2} \tag{22}$$

where $u_{t-2} = \hat{m}_{t-2} \oslash (\sqrt{\hat{v}_{t-2}} + \varepsilon)$ is the previous ADAM update direction and β_h is the hypergradient learning rate. Intuitively, if the current gradient aligns with the previous update vector ($\nabla f(w_{t-1})^T u_{t-2} > 0$), the optimizer increases α_t to accelerate progress along the valley. Conversely, if the vectors oppose each other, it decreases α_t to prevent overshooting. On the 100D Extended Rosenbrock problem, this adaptive mechanism reduced the required iterations from 15,094 to 8,736 (a 42.1% reduction) while reaching the exact same local minimum.



(a) Standard ADAM (15,094 iterations)



(b) ADAM-DG (8,736 iterations)

Figure 9: Comparison of objective function convergence on logarithmic scale between Standard ADAM and ADAM with Double Gradient (Hypergradient Descent) on the 100D Extended Rosenbrock benchmark.

6.2 Stabilized Curvature Preconditioning

While first-order ADAM uses coordinate-wise scaling $\sqrt{\hat{v}_t}$, it treats parameter dimensions independently. Curvature preconditioning transforms the gradient by an approximation of the inverse Hessian matrix before computing moment estimates:

$$p_t = H^{-1}(w_t)\nabla f(w_t) \tag{23}$$

On non-convex objective landscapes, raw Newton steps can diverge because the Hessian $H(w)$ is not positive definite away from stationary points, causing $H^{-1}\nabla f$ to point uphill. To ensure numerical stability, our implementation incorporates two safeguards based on Nocedal and Wright [8]:

1. **Adaptive Diagonal Regularization:** We compute the Cholesky factorization of $H + \tau I$. If factorization fails (indicating non-positive definiteness), the regularization parameter τ is iteratively increased by factors of 10 until positive definiteness is restored.

2. **Descent Direction Verification:** Before updating, we check the condition $\nabla f(w_t)^T p_t > 0$. If this test fails, the algorithm discards the preconditioned direction and falls back to the steepest descent gradient $\nabla f(w_t)$.

To avoid analytical second derivatives, the Hessian entries are estimated numerically via central differences of the gradient. To maintain computational efficiency, the preconditioning matrix is refreshed every 50 iterations while testing the descent direction condition at every step, reducing computation time to 0.91 seconds.

6.3 Black-Box / Zeroth-Order Gradient Estimation

For optimization problems where analytical gradients are unavailable (e.g., external simulation software or experimental systems), the gradient vector can be approximated using function evaluations alone:

- **Central Finite Differences (CFD):** Evaluates symmetric perturbations along each coordinate:
$$g_i \approx \frac{f(w + he_i) - f(w - he_i)}{2h}, \quad i = 1, \ldots, D \tag{24}$$
CFD achieves $\mathcal{O}(h^2)$ truncation accuracy but requires $2D$ function calls per step.

- **Simultaneous Perturbation Stochastic Approximation (SPSA):** Perturbs all D variables simultaneously with a random vector $\Delta \in \mathbb{R}^D$ (e.g., Rademacher ± 1):
$$\hat{g} \approx \frac{f(w + h\Delta) - f(w - h\Delta)}{2h} \Delta^{-1} \tag{25}$$
SPSA requires only 2 function evaluations per step regardless of dimension D. The stochastic perturbation noise is subsequently smoothed by ADAM's momentum accumulator m_t.

6.4 Twin-Gradient Step-Length Modulation

Standard ADAM updates a single solution vector, using exponential moving averages to move through narrow ravines. Inspired by De Magistris et al. [7], we implement a dual-trajectory version with two iterates, w_1 and w_2, initialized from different points. Unlike the full framework in [7], which uses Adaptive Barzilai–Borwein ($\text{ABB}_{\min}$) steps, damping, and orthogonal initialization, our implementation couples standard ADAM directions through a closed-form distance-minimizing step multiplier clamped to $[0.5, 3.5]$.

Algorithm Formulation: At iteration t, each trajectory independently computes its gradient and nominal ADAM step vector:

$$s_1 = -\alpha \frac{\hat{m}_1}{\sqrt{\hat{v}_1} + \varepsilon}, \qquad s_2 = -\alpha \frac{\hat{m}_2}{\sqrt{\hat{v}_2} + \varepsilon} \tag{26}$$

Instead of applying the fixed stepsize α along both directions, scalar multipliers γ_1, γ_2 are chosen at every iteration to minimize the squared Euclidean distance between the next positions:

$$\min_{\gamma_1, \gamma_2} \frac{1}{2} \left\| (w_1 + \gamma_1 s_1) - (w_2 + \gamma_2 s_2) \right\|_2^2 \tag{27}$$

Setting the partial derivatives with respect to γ_1 and γ_2 to zero yields the 2×2 symmetric linear system:

$$\begin{bmatrix} s_1^T s_1 & -s_1^T s_2 \\ -s_1^T s_2 & s_2^T s_2 \end{bmatrix} \begin{bmatrix} \gamma_1 \\ \gamma_2 \end{bmatrix} = \begin{bmatrix} -s_1^T \delta \\ s_2^T \delta \end{bmatrix}, \qquad \text{with } \delta = w_1 - w_2 \tag{28}$$

Solving this 2×2 Gram system requires only vector inner products, adding $\mathcal{O}(D)$ operations per step.

Safeguards and Clamp Behavior: Two safeguards stabilize the coupling without an objective line search:

1. **Collinearity check:** We compute $\det_{\text{norm}} = 1 - \cos^2\theta$, where θ is the angle between s_1 and s_2. When search directions become nearly parallel ($\det_{\text{norm}} \leq 10^{-6}$), the Gram matrix is ill-conditioned and updates revert to standard uncoupled steps ($\gamma_1 = \gamma_2 = 1$).

2. **Step-multiplier clamp:** We restrict the multipliers to $\gamma_1, \gamma_2 \in [0.5, 3.5]$. Without this bound, the unconstrained multiplier is huge at the start, because the nominal ADAM step is tiny compared with the distance between the twins, and the iterates jump to the point where the two search rays meet, far uphill. In practice, the clamp is active in almost every iteration: in 2D the unconstrained multiplier exceeds 3.5 in 97% of steps, and in 100D it falls below 0.5 in 91% of steps. The step actually taken is therefore about 3.5 (in 2D) or 0.5 (in 100D) times the nominal ADAM step, not the distance-minimizing one.

Algorithm 2 Twin-Modulated ADAM Optimizer

Require: Objective gradient $\nabla f(w)$, initial vectors w_{01}, w_{02}, max iterations K, hyperparameters $(\alpha, \beta_1, \beta_2, \varepsilon, \tau)$

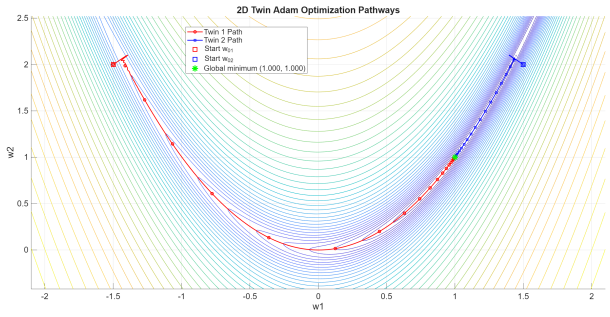
$w_1 \leftarrow w_{01},\ w_2 \leftarrow w_{02},\ m_1, m_2 \leftarrow \mathbf{0},\ v_1, v_2 \leftarrow \mathbf{0},\ t \leftarrow 0$
while $\|\nabla f(w_1)\|_2 > \tau$ **and** $\|\nabla f(w_2)\|_2 > \tau$ **and** $t < K$ **do**
  $t \leftarrow t + 1$
  for $k \in \{1, 2\}$ **do**
    $g_k \leftarrow \nabla f(w_k)$
    $m_k \leftarrow \beta_1 m_k + (1 - \beta_1) g_k, \quad v_k \leftarrow \beta_2 v_k + (1 - \beta_2)(g_k \odot g_k)$ ▷ Moment updates
    $\hat{m}_k \leftarrow m_k / (1 - \beta_1^t), \quad \hat{v}_k \leftarrow v_k / (1 - \beta_2^t)$ ▷ Bias correction
    $s_k \leftarrow -\alpha \cdot \hat{m}_k \oslash (\sqrt{\hat{v}_k} + \varepsilon)$ ▷ Nominal ADAM step
  end for
  $\delta \leftarrow w_1 - w_2, \quad \det \leftarrow \|s_1\|_2^2 \|s_2\|_2^2 - (s_1^T s_2)^2$
  if $\det / (\|s_1\|_2^2 \|s_2\|_2^2) > 10^{-6}$ **then**
    $\gamma_1 \leftarrow \frac{-(s_1^T \delta)\|s_2\|_2^2 + (s_1^T s_2)(s_2^T \delta)}{\det}, \quad \gamma_2 \leftarrow \frac{(s_2^T \delta)\|s_1\|_2^2 - (s_1^T s_2)(s_1^T \delta)}{\det}$
    $\gamma_1 \leftarrow \min(\max(\gamma_1, 0.5), 3.5), \quad \gamma_2 \leftarrow \min(\max(\gamma_2, 0.5), 3.5)$ ▷ Step bounds
  else
    $\gamma_1 \leftarrow 1.0, \quad \gamma_2 \leftarrow 1.0$ ▷ Collinear fallback
  end if
  $w_1 \leftarrow w_1 + \gamma_1 s_1, \quad w_2 \leftarrow w_2 + \gamma_2 s_2$ ▷ State update
  history.w1$(:, t) \leftarrow w_1$, history.w2$(:, t) \leftarrow w_2$
end while
return $w^* = \arg\min_{w \in \{w_1, w_2\}} \|\nabla f(w)\|_2$, history

Convergence Dynamics on 2D and 100D Benchmarks:

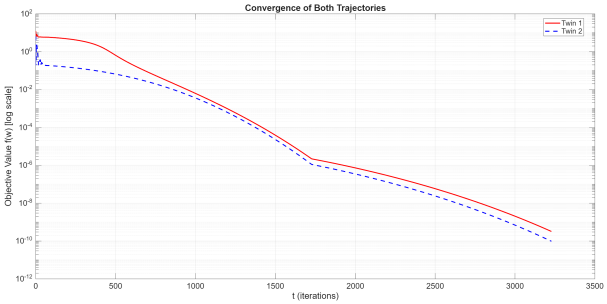
- **2D Rosenbrock Benchmark:** Twin 1 starts at $(-1.5, 2.0)$ and Twin 2 at $(1.5, 2.0)$ (Figure 10(a)). Both enter the curved valley and come within $\|w_1 - w_2\|_2 < 0.05$ at iteration

1,378 near $(1, 1)$. Twin 2 reaches tolerance $\tau = 10^{-5}$ at iteration 3,228 (gradient norm 9.999×10^{-6}; Twin 1 at 1.8×10^{-5}), with both twins reaching $f \leq 10^{-9}$ (Figure 10(b)). Because γ is clamped at 3.5 for 97% of steps, the effective learning rate is $\alpha_{\text{eff}} \approx 0.035$. Plain single ADAM with $\alpha = 0.035$ converges in 2,203 iterations (w_{01}) and 2,160 iterations (w_{02}), showing that the 22.1% iteration reduction (4,145 → 3,228) stems from the larger effective step size rather than coupling geometry. Due to two gradients per step, Twin ADAM required 6,456 evaluations—55.8% more than single ADAM from w_{01} (4,145 evals; 4,016 from w_{02}).

- **100D Extended Rosenbrock Benchmark:** The separation $\|w_1 - w_2\|_2$ drops from 35.0 to about 1.6 in the first 600 iterations, then stays between about 1.5 and 2.1 (Figure 11(b)). Twin 1 moves toward the local minimum (first coordinate $w_{1,1} \approx -0.9933$, $f \approx 3.9866$), reaching $\|\nabla f(w_1)\|_2 = 9.90 \times 10^{-6} \leq 10^{-5}$ at iteration 14,921 and stopping the run. Twin 2 enters the global basin ($f = 1.24 \times 10^{-7}$), but its gradient norm is 4.40×10^{-4}, so the optimizer returns Twin 1 as w^* based on smaller gradient norm. Plain ADAM from w_{02} also enters the global basin ($f = 2.15 \times 10^{-3}$ at 25,000 iterations), suggesting this is mainly due to the starting point w_{02} rather than the twin coupling. In total, Twin ADAM used 29,842 gradient evaluations and took 3× longer (0.33 s vs. 0.11 s).

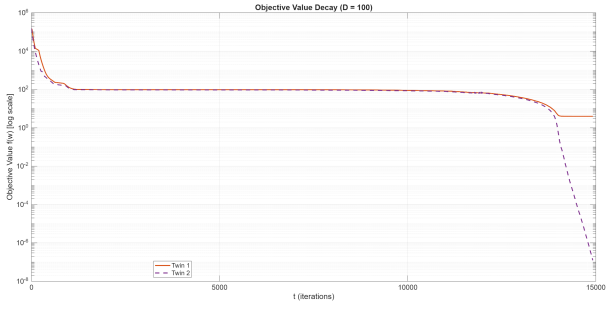


(a) Optimization trajectories on 2D Rosenbrock

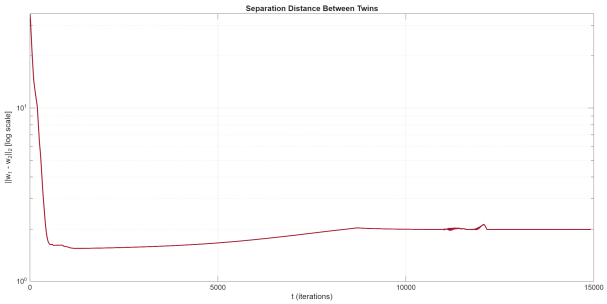


(b) Objective function convergence history $f(w_1), f(w_2)$

Figure 10: Twin ADAM convergence on the 2D Rosenbrock benchmark initialized from opposite sides of the valley ($w_{01} = (-1.5, 2.0)$, $w_{02} = (1.5, 2.0)$).



(a) Objective function convergence history ($D = 100$)



(b) Inter-twin trajectory distance $\|w_1 - w_2\|_2$

Figure 11: Twin ADAM performance on the 100D Extended Rosenbrock benchmark, displaying mutual distance contraction into the 100-dimensional valley.

7 High-Dimensional Application: ANN Training

7.1 Motivation

The benchmarks in Sections 3 to 5 have at most 100 variables and a known minimum. Practical problems usually have far more unknowns, so in this section the ADAM variants are used to train an artificial neural network (ANN). The network for the 100-dimensional problem, with layers $[100, 128, 64, 1]$, already has $P = \sum_l n_l(n_{l-1}+1) = 21\,249$ parameters. Their gradient is obtained with the adjoint method at about the cost of one forward pass, while central finite differences would need $2P = 42\,498$ loss evaluations. Since the loss of a network has no known minimizer, performance is measured by the prediction error on unseen data instead of the distance to w^*.

7.2 Network and Adjoint Gradient

For layers $l = 1, \dots, L$ with weights $\boldsymbol{W}^{(l)}$ and biases $\boldsymbol{b}^{(l)}$, and samples stored column-wise in $\boldsymbol{A}^{(0)} = \boldsymbol{X}$, the forward pass and the loss are

$$\boldsymbol{Z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{A}^{(l-1)} + \boldsymbol{b}^{(l)}, \qquad \boldsymbol{A}^{(l)} = \sigma\big(\boldsymbol{Z}^{(l)}\big), \qquad J = \frac{1}{n_L N_b}\big\|\boldsymbol{A}^{(L)} - \tilde{\boldsymbol{Y}}\big\|_F^2, \tag{29}$$

with $\sigma = \tanh$ in the hidden layers, a linear output layer and standardized targets $\tilde{\boldsymbol{Y}}$. The gradient is computed by reverse-mode differentiation, known as backpropagation [11] or the discrete adjoint method [2]. The adjoint $\boldsymbol{\Delta}^{(l)} = \partial J / \partial \boldsymbol{Z}^{(l)}$ is passed backwards from the output (Figure 12):

$$\boldsymbol{\Delta}^{(L)} = \frac{2}{n_L N_b}\big(\boldsymbol{A}^{(L)} - \tilde{\boldsymbol{Y}}\big), \qquad \boldsymbol{\Delta}^{(l-1)} = \big(\boldsymbol{W}^{(l)\mathsf{T}}\boldsymbol{\Delta}^{(l)}\big) \odot \sigma'\big(\boldsymbol{Z}^{(l-1)}\big), \tag{30}$$

$$\frac{\partial J}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\Delta}^{(l)}\boldsymbol{A}^{(l-1)\mathsf{T}}, \qquad \frac{\partial J}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\Delta}^{(l)}\mathbf{1}. \tag{31}$$

The gradients have the same structure as the parameters, so the optimizer step functions of the previous sections are used without changes.

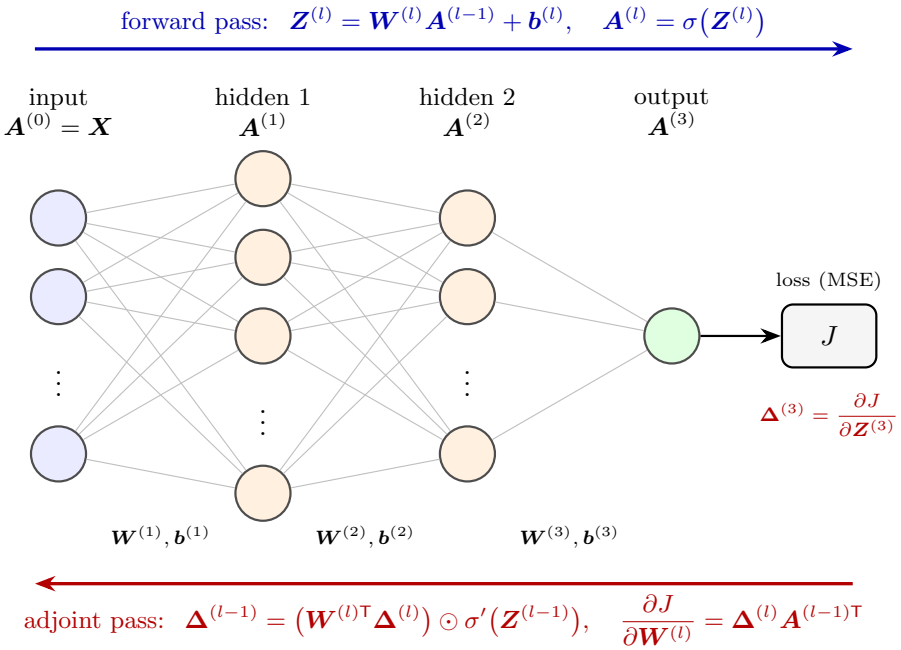


Figure 12: Fully connected network with forward pass (left to right) and adjoint pass (right to left).

Gradient check. The adjoint gradient was compared with central finite differences, $g_j^{\mathrm{FD}} = [J(\boldsymbol{\theta} + h\boldsymbol{e}_j) - J(\boldsymbol{\theta} - h\boldsymbol{e}_j)]/2h$ with $h = 10^{-6}$, for every weight and bias of a small network. The relative error $|g_j^{\mathrm{FD}} - g_j^{\mathrm{adj}}|/\max(1, |g_j^{\mathrm{FD}}|)$ stayed below 2.5×10^{-9} for all parameter arrays (Table 8).

Table 8: Gradient check: maximum relative error between adjoint and central finite-difference gradients (layers $[1, 5, 4, 1]$, tanh, $h = 10^{-6}$).

Parameter array	**Size**	**Max. relative error** e	**Passed** ($e < 10^{-5}$)
$\boldsymbol{W}^{(1)}$	5×1	1.27×10^{-9}	yes
$\boldsymbol{b}^{(1)}$	5×1	2.44×10^{-9}	yes
$\boldsymbol{W}^{(2)}$	4×5	1.94×10^{-9}	yes
$\boldsymbol{b}^{(2)}$	4×1	1.25×10^{-9}	yes
$\boldsymbol{W}^{(3)}$	1×4	3.07×10^{-10}	yes
$\boldsymbol{b}^{(3)}$	1×1	3.03×10^{-11}	yes

7.3 Training Setup

The data are split randomly into 70% training, 15% validation and 15% test samples. Inputs are scaled to $[-1, 1]$ and outputs standardized with training statistics, so a scaled MSE of 1 equals predicting the mean. The batch size B selects full-batch training ($B = N_{\mathrm{tr}}$), mini-batch training ($1 < B < N_{\mathrm{tr}}$) or SGD ($B = 1$); all results below use full batch. The weights with the lowest validation loss are kept and evaluated once on the test set (Algorithm 3).

ADAM, AdamW [6], AMSGrad [9] and Twin ADAM use the same training code; only the step function is exchanged. All runs use the same seed, so data, split and initial weights are identical, and the same hyperparameters (Table 9). The optimizers are compared by the test MSE (main criterion), the training and best validation MSE, the generalisation ratio $G = J_{\mathrm{te}}/J_{\mathrm{tr}}$ ($G \gg 1$ indicates overfitting) and the wall-clock time. The computational cost of E epochs is approximately

$$C \approx 6\, P\, k\, E\, N_{\mathrm{tr}}, \tag{32}$$

since a forward and an adjoint pass need about $2P$ and $4P$ operations per sample, with $k = 1$ for ADAM, AdamW and AMSGrad and $k = 2$ for Twin ADAM.

Table 9: Test problems and common settings.

Test problems	$\sin(0.2x)$ with noise (1D); Rosenbrock fit with $D = 2$ and 100 inputs; Concrete Compressive Strength data set (UCI)
Network (100D)	$[100, 128, 64, 1]$, $P = 21\,249$, $N = 100\,000$ samples in $[-2, 2]^{100}$
Activation	tanh (hidden), linear (output)
Hyperparameters	$\alpha = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, AdamW $\lambda = 0.1$
Batch size	full batch ($B = N_{\mathrm{tr}}$)
Seed	1, reset before every run

Twin ADAM. The twin-gradient method of Section 6.4 [7] is applied to two independently initialised networks $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ trained on the same batches. Their ADAM steps $\boldsymbol{s}_1, \boldsymbol{s}_2$ are scaled by

$$\min_{\gamma_1, \gamma_2} \left\| (\boldsymbol{\theta}_1 + \gamma_1 \boldsymbol{s}_1) - (\boldsymbol{\theta}_2 + \gamma_2 \boldsymbol{s}_2) \right\|_2^2, \qquad \gamma_{1,2} \in [0.5,\, 3.5], \tag{33}$$

with $\gamma_1 = \gamma_2 = 1$ if the steps are nearly parallel. After each epoch the network with the lower validation loss is kept. Each update needs two gradient evaluations.

Algorithm 3 ANN training with an exchangeable ADAM-type optimizer (`ADAM_ANN.m`)

Require: Data $(\boldsymbol{X}, \boldsymbol{Y})$, layer widths, activation σ, epochs E, batch size B, step function `opt`, hyperparameters $(\alpha, \beta_1, \beta_2, \varepsilon)$

Split data 70/15/15; scale inputs to $[-1, 1]$, standardize outputs (training statistics)
Initialise $\boldsymbol{\theta}$; optimizer state $\leftarrow \emptyset$; $t \leftarrow 0$; $J_{\text{va}}^{\text{best}} \leftarrow \infty$
for $e = 1, \ldots, E$ **do**
    Shuffle the training samples
    for each batch $\mathcal{B}$ of size B **do**
        $\boldsymbol{A}^{(L)} \leftarrow$ forward pass on $\mathcal{B}$ ▷ store $\boldsymbol{Z}^{(l)}, \boldsymbol{A}^{(l)}$
        $\nabla_{\boldsymbol{\theta}} J \leftarrow$ adjoint pass on $\mathcal{B}$ ▷ Section 7.2
        $t \leftarrow t + 1$
        $[\boldsymbol{\theta}, \text{state}] \leftarrow \texttt{opt}(\boldsymbol{\theta}, \nabla_{\boldsymbol{\theta}} J, \text{state}, t, \text{hp})$ ▷ ADAM, AdamW, AMSGrad
    end for
    Evaluate J_{tr} and J_{va} on the complete sets
    if $J_{\text{va}} < J_{\text{va}}^{\text{best}}$ **then**
        $J_{\text{va}}^{\text{best}} \leftarrow J_{\text{va}}$; $\boldsymbol{\theta}^{\text{best}} \leftarrow \boldsymbol{\theta}$
    end if
end for
Evaluate J_{te} with $\boldsymbol{\theta}^{\text{best}}$
return $\boldsymbol{\theta}^{\text{best}}$, loss histories, J_{te}

7.4 Results

Table 10 summarises all ANN benchmarks.

Table 10: ANN benchmark results (scaled MSE, seed 1, settings of Table 9).

Optimizer	Train MSE	Best val. MSE	Test MSE	G	Time [s]
1D: $\sin(0.2x)$ *with noise*					
ADAM	4.67×10^{-3}	1.32×10^{-2}	1.20×10^{-2}	2.56	2.99
AdamW	1.64×10^{-1}	1.15×10^{-1}	1.18×10^{-1}	0.72	3.80
AMSGrad	2.34×10^{-1}	3.24×10^{-1}	3.19×10^{-1}	1.37	3.91
Twin ADAM	9.42×10^{-3}	1.36×10^{-2}	1.30×10^{-2}	1.38	6.99
Rosenbrock 2D					
ADAM	5.16×10^{-5}	1.48×10^{-4}	4.70×10^{-5}	0.91	1.99
AdamW	2.21×10^{-3}	5.04×10^{-3}	2.54×10^{-3}	1.14	1.98
AMSGrad	1.02×10^{-4}	3.07×10^{-4}	9.36×10^{-5}	0.92	1.95
Twin ADAM	8.31×10^{-5}	2.50×10^{-4}	8.39×10^{-5}	1.0	3.2
Rosenbrock 100D					
ADAM	2.01×10^{-1}	5.44×10^{-1}	5.44×10^{-1}	2.70	493.18
AdamW	2.95×10^{-1}	6.27×10^{-1}	6.44×10^{-1}	2.18	531.00
AMSGrad	1.94×10^{-1}	5.26×10^{-1}	5.23×10^{-1}	2.69	573.19
Twin ADAM	5.21×10^{-1}	7.12×10^{-1}	7.25×10^{-1}	1.39	950.16
Real-world data set: Concrete dataset					
ADAM	4.68×10^{-2}	9.23×10^{-2}	1.15×10^{-1}	2.46	3.37
AdamW	4.20×10^{-2}	7.61×10^{-2}	1.13×10^{-1}	2.71	3.34
AMSGrad	8.51×10^{-2}	1.05×10^{-1}	1.42×10^{-1}	1.67	3.56
Twin ADAM	4.83×10^{-2}	7.48×10^{-2}	1.53×10^{-1}	3.17	6.04

Low-dimensional fits. The 1D and 2D cases check the complete pipeline: the prediction follows $\sin(0.2x)$ (Figure 13) and reproduces the curved valley of the 2D Rosenbrock function (Figure 14). ADAM gives the lowest test MSE in both cases. The test MSE of AdamW is about 10 times (1D) and 54 times (2D) higher, and that of AMSGrad about 27 times higher in 1D.

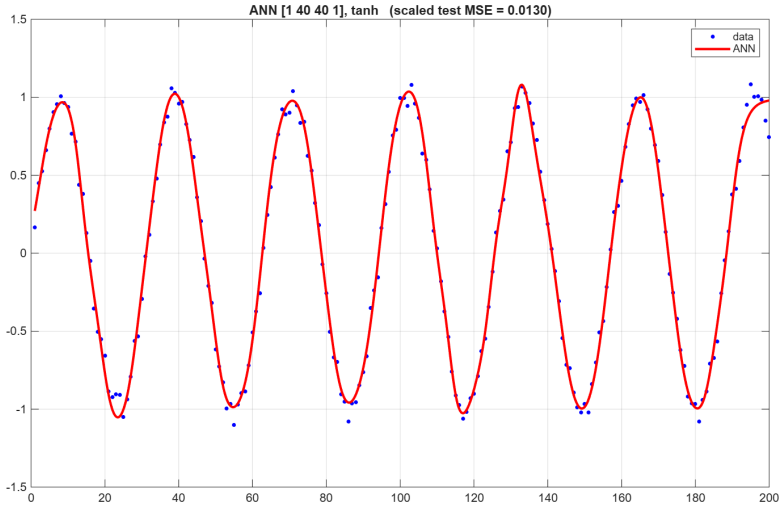
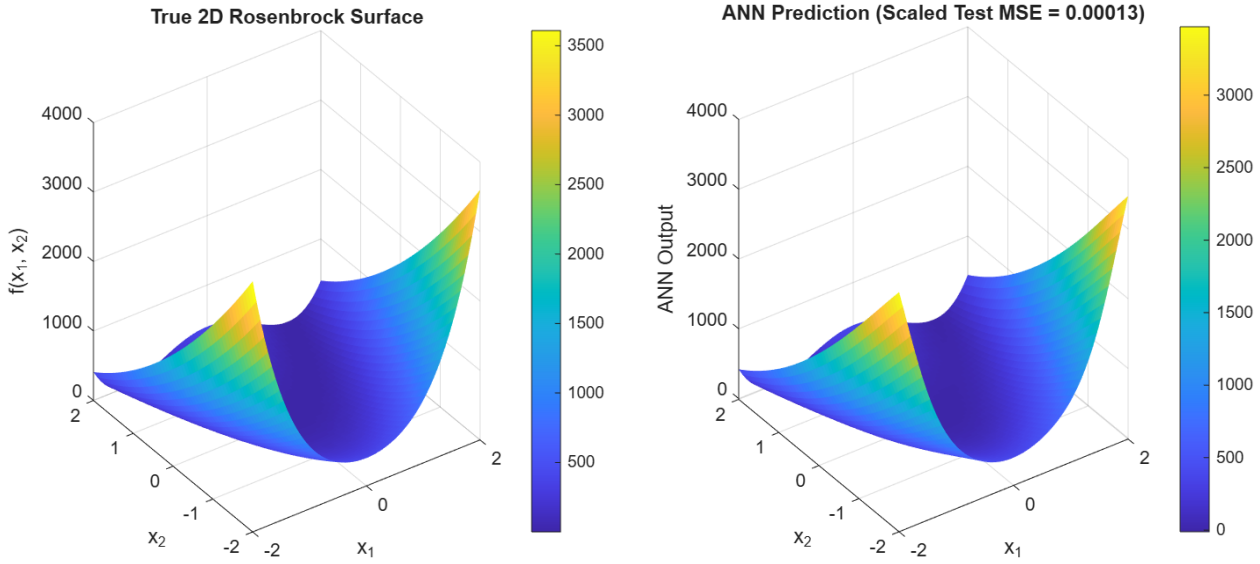


Figure 13: ANN approximation of $\sin(0.2x)$ with noise (ADAM, settings of Table 9).

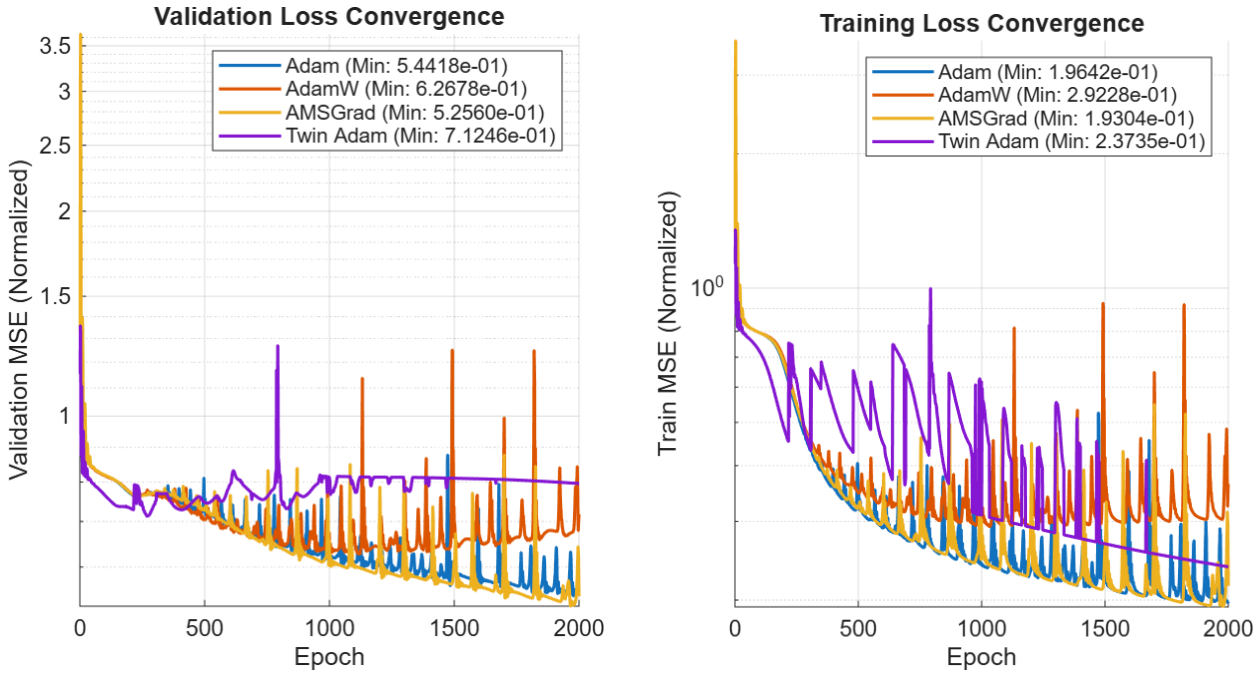


(a) Original Function (b) ANN prediction

Figure 14: ANN approximation of the 2D Rosenbrock function (ADAM, settings of Table 9).

100-dimensional Rosenbrock. Twin ADAM has the lowest validation loss between about epoch 100 and 250 but then stagnates at about 0.71 (Figure 15). ADAM, AdamW and AMSGrad show repeated spikes after about epoch 300, largest for AdamW. AMSGrad reaches the lowest test MSE (5.23×10^{-1}), followed by ADAM (5.44×10^{-1}). With G between 1.39 and 2.70, all optimizers overfit to some degree despite $N = 100\,000$ samples.

Concrete data set. The Concrete Compressive Strength data set [13] contains 1030 laboratory tests with 8 inputs (seven mixture components in kg/m^3 and the age in days) and the compressive strength in MPa as output. AdamW reaches the lowest test MSE. The validation loss of ADAM



(a) Validation loss

(b) Training loss

Figure 15: Convergence of ADAM, AdamW, AMSGrad and Twin ADAM for the 100-dimensional Rosenbrock regression.

and AMSGrad rises again after about 6500 and 7000 epochs, which shows the onset of overfitting (Figure 16).

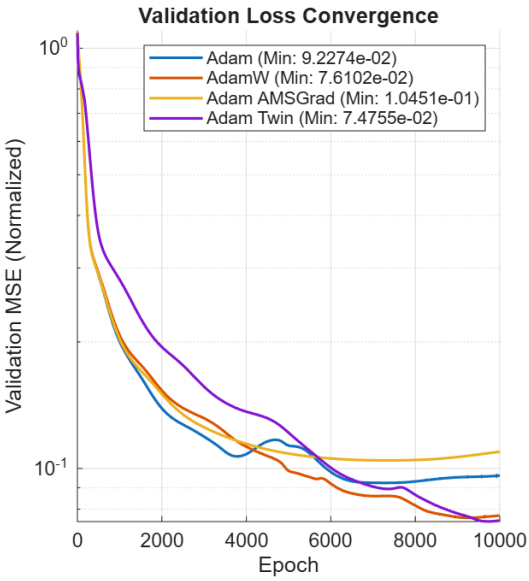


Figure 16: Validation loss of ADAM, AdamW, AMSGrad and Twin ADAM over the epochs for the Concrete Compressive Strength data set.

Twin ADAM. Twin ADAM did not reach a lower validation loss than ADAM on the 1D, 2D and 100D problems. On the Concrete data its validation loss was lower (7.48×10^{-2} against 9.23×10^{-2}), but its test MSE was the highest. Its time per epoch was 1.6 to 2.3 times that of ADAM (Figure 17).

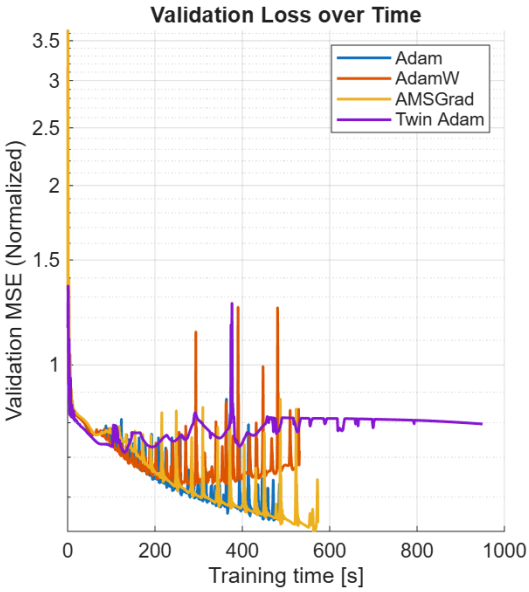


Figure 17: ADAM and Twin ADAM compared at equal computing time (100-dimensional Rosenbrock regression).

7.5 Discussion

The adjoint gradient agreed with finite differences to about 10^{-9} and made training with 21 249 unknowns practical. ADAM was best on the 1D and 2D fits and within about 4% of the best elsewhere, AMSGrad was best on the 100D problem and AdamW on the Concrete data. The ranking of the direct Rosenbrock problem (Section 5.4), where AdamW was best and AMSGrad worst, therefore does not carry over to network training. Twin ADAM doubled the cost per update without improving the networks, although it can help on the single-basin problems of Section 6.4. Since all runs use one seed and the same hyperparameters, repeated runs and a learning-rate study would show how robust this ranking is.

8 Future Research and Development Plans

We plan to extend our MATLAB optimization framework in the following directions:

- **Stochastic Mini-Batch Analysis:** Benchmarking ADAM-DG and AdamW under stochastic noise to evaluate variance adaptation on non-deterministic loss landscapes.
- **High-Dimensional Scaling ($D = 10^3 \dots 10^5$):** Implementing matrix-free Hessian-vector products ($Hv \approx \frac{\nabla f(w+\delta v) - \nabla f(w)}{\delta}$) to capture directional curvature at large dimensions without forming full matrices.
- **Neural Network Training with Adjoint Gradients:** Train linear and non-linear networks in MATLAB, using LU decomposition ($\mathbf{A} = \mathbf{LU}$) and the adjoint state method to compute gradients efficiently for regression and MNIST.
- **Uncertainty Quantification in Optimization:** Linking ADAM's adaptive variance vector v_t with stochastic sensitivity estimation for structural reliability problems.

References

[1] Atilim Gunes Baydin, Robert Cornish, David Martinez-Rubio, Mark Schmidt, and Frank Wood. Online learning rate adaptation with hypergradient descent. In *International Conference on Learning Representations (ICLR)*, 2018.

[2] Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.

[3] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.

[4] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[5] S. Kok and C. Sandrock. Locating and characterizing the stationary points of the extended Rosenbrock function. *Evolutionary Computation*, 17(3):437–453, 2009.

[6] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization (AdamW). *arXiv preprint arXiv:1711.05101*, 2017.

[7] Anna De Magistris, Michiel E. Hochstenbach, and Gerardo Toraldo. A twin gradient method for unconstrained optimization. *arXiv preprint arXiv:2607.11575*, 2026.

[8] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.

[9] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of Adam and beyond. In *International Conference on Learning Representations (ICLR)*, 2018.

[10] Howard H. Rosenbrock. An automatic method for finding the greatest or least value of a function. *The Computer Journal*, 3(3):175–184, 1960.

[11] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[12] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012.

[13] I-Cheng Yeh. Modeling of strength of high-performance concrete using artificial neural networks. *Cement and Concrete Research*, 28(12):1797–1808, 1998. Data set: UCI Machine Learning Repository, `https://doi.org/10.24432/C5PK67`.